SEV 플랫폼 초기화

개요

AMD SEV 플랫폼 초기화는 보안 가상화 환경을 구축하기 위한 첫 번째 단계입니다. 이 과정에서 플랫폼의 보안 상태를 설정하고, 필요한 암호화 키를 생성하며, 신뢰할 수 있는 실행 환경을 준비합니다.

SEV 플랫폼 초기화 단계별 프로세스

1. 플랫폼 상태 확인

초기화를 시작하기 전에 현재 플랫폼의 상태를 확인해야 합니다:

```
use sev::firmware::host::{Firmware, State};


let mut firmware = Firmware::open()?;
let status = firmware.platform_status()?;


match status.state {
State::Uninitialized => {
println!("플랫폼이 초기화되지 않음 - 초기화 진행 필요");
}
State::Initialized => {
println!("플랫폼이 이미 초기화됨 - 게스트 실행 준비 완료");
```

```
}
State::Working => {
println!("플랫폼이 작동 중 - 게스트 VM이 실행 중");
}
}
```

2. 플랫폼 리셋 (필요시)

기존 상태를 초기화하고 새로운 설정을 적용하려면 플랫폼을 리셋해야 합니다:

```
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// 플랫폼 리셋 - 모든 영구 상태 초기화
firmware.platform_reset()?;
println!("플랫폼 리셋 완료");

// 리셋 후 상태 확인
let status = firmware.platform_status()?;
assert_eq!(status.state, State::Uninitialized);
```

3. Platform Encryption Key (PEK) 생성

PEK는 플랫폼의 기본 암호화 키로, 모든 보안 작업의 기반이 됩니다:

```rust
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// PEK 생성
firmware.pek_generate()?;
println!("PEK 생성 완료");

// PEK 인증서 서명 요청 생성
let pek_cert = firmware.pek_csr()?;
println!("PEK CSR 생성 완료");
```

4. Platform Diffie-Hellman (PDH) 키 생성

PDH 키는 보안 세션 설정을 위한 키 교환에 사용됩니다:

```rust
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// PDH 키 쌍 생성
firmware.pdh_generate()?;
println!("PDH 키 생성 완료");
```

5. 인증서 체인 내보내기

생성된 키들의 인증서 체인을 내보내어 검증할 수 있습니다:

```rust
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// 인증서 체인 내보내기
let chain = firmware.pdh_cert_export()?;

println!("PDH 인증서: {:?}", chain.pdh);
println!("PEK 인증서: {:?}", chain.pek);
println!("OCA 인증서: {:?}", chain.oca);
println!("CEK 인증서: {:?}", chain.cek);
```

6. 초기화 완료 확인

모든 초기화 단계가 완료되었는지 확인합니다:

```rust
use sev::firmware::host::{Firmware, State};

let mut firmware = Firmware::open()?;
let status = firmware.platform_status()?;
```

```
if status.state == State::Initialized {
println!("플랫폼 초기화 성공적으로 완료");
println!("펌웨어 버전: {}.{}",
status.build.version.major,
status.build.version.minor);
println!("활성 게스트 수: {}", status.guests);
} else {
println!("초기화가 완료되지 않음: {:?}", status.state);
}
```

완전한 초기화 프로세스 예제

다음은 전체 초기화 프로세스를 수행하는 완전한 예제입니다:

```
use sev::firmware::host::{Firmware, State};
use sev::error::UserApiError;

fn initialize_sev_platform() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()?;
println!("=== SEV 플랫폼 초기화 시작 ===");
// 1. 현재 상태 확인
let initial_status = firmware.platform_status()?;
println!("초기 플랫폼 상태: {:?}", initial_status.state);
// 2. 필요시 플랫폼 리셋
```

```
if initial_status.state != State::Uninitialized {
println!("플랫폼 리셋 수행 중...");
firmware.platform_reset()?;
println!("플랫폼 리셋 완료");
}
// 3. PEK 생성
println!("PEK 생성 중...");
firmware.pek_generate()?;
println!("PEK 생성 완료");
// 4. PEK 인증서 서명 요청
println!("PEK CSR 생성 중...");
let _pek_cert = firmware.pek_csr()?;
println!("PEK CSR 생성 완료");
// 5. PDH 키 생성
println!("PDH 키 생성 중...");
firmware.pdh_generate()?;
println!("PDH 키 생성 완료");
// 6. 인증서 체인 내보내기
println!("인증서 체인 내보내기 중...");
let chain = firmware.pdh_cert_export()?;
println!("인증서 체인 내보내기 완료");
// 7. 최종 상태 확인
let final_status = firmware.platform_status()?;
```

```
println!("최종 플랫폼 상태: {:?}", final_status.state);

if  final_status.state == State::Initialized {

println!("=== SEV 플랫폼 초기화 성공 ===");

println!("펌웨어 버전: {}.{}",

final_status.build.version.major,

final_status.build.version.minor);

} else {

println!("초기화 실패: 예상하지 못한 상태 {:?}", final_status.state);

}

Ok(())

}
```

플랫폼 소유권 설정 과정

소유권 개념

SEV 플랫폼 소유권은 누가 플랫폼을 제어할 수 있는지를 결정합니다. 소유권 설정을 통해 신뢰할 수 있는 인증 기관(OCA)을 지정할 수 있습니다.

소유권 설정 단계

1. OCA (Owner Certificate Authority) 생성

```
use sev::certs::sev::{sev::Certificate, sev::Usage, Signer};

// OCA 인증서와 개인키 생성
```

```rust
let (mut oca_cert, oca_private_key) = Certificate::generate(Usage::OCA)?;

// 자체 서명
oca_private_key.sign(&mut oca_cert)?;
println!("OCA 인증서 생성 및 서명 완료");
```

2. PEK 인증서 서명

```rust
use sev::firmware::host::Firmware;

let mut firmware = Firmware::open()?;

// PEK CSR 요청
let mut pek_cert = firmware.pek_csr()?;

// OCA 개인키로 PEK 인증서 서명
oca_private_key.sign(&mut pek_cert)?;
println!("PEK 인증서 서명 완료");
```

3. 인증서 가져오기

```rust
// 서명된 PEK와 OCA 인증서를 플랫폼에 가져오기
firmware.pek_cert_import(&pek_cert, &oca_cert)?;
```

```
println!("인증서 가져오기 완료 - 소유권 설정됨");
```

4. 소유권 설정 검증

```
// 인증서 체인 내보내기로 소유권 확인
let chain = firmware.pdh_cert_export()?;

// OCA 인증서가 올바르게 설정되었는지 확인
assert_eq!(chain.oca, oca_cert);
println!("소유권 설정 검증 완료");
```

SEV-SNP 플랫폼 초기화

SEV-SNP는 추가적인 초기화 단계가 필요합니다:

SNP 플랫폼 상태 확인

```
use sev::firmware::host::{Firmware, SnpPlatformStatus};

let mut firmware = Firmware::open()?;

// SNP 플랫폼 상태 조회
let snp_status: SnpPlatformStatus = firmware.snp_platform_status()?;

println!("SNP 펌웨어 버전: {}.{}", snp_status.version.0, snp_status.version.1);
```

```
println!("빌드 ID: {}", snp_status.build_id);
println!("플랫폼 상태: {}", snp_status.state);
println!("TCB 버전: {:?}", snp_status.platform_tcb_version);
```

SNP 설정 적용

```
use sev::firmware::host::{Firmware, Config, TcbVersion, MaskId};

let mut firmware = Firmware::open()?;

// SNP 설정 생성
let config = Config::new(
TcbVersion::new(Some(3), 0, 10, 169, 0), // TCB 버전
MaskId(0), // 마스크 ID
);

// 설정 적용
firmware.snp_set_config(config)?;
println!("SNP 설정 적용 완료");
```

SNP 커밋

```
// SNP 설정을 커밋하여 영구적으로 적용
```

```
firmware.snp_commit()?;

println!("SNP 커밋 완료");
```

초기화 과정에서 발생할 수 있는 오류 및 해결 방법

1. 권한 오류

```
use sev::error::{UserApiError};

use std::io::ErrorKind;


match  firmware.platform_status() {

Err(UserApiError::IOError(io_err)) if  io_err.kind() == ErrorKind::PermissionDen

println!("오류: 권한이 부족합니다.");

println!("해결: sudo 권한으로 실행하거나 사용자를 적절한 그룹에 추가하세요.");

}

Err(e) => println!("기타 오류: {:?}", e),

Ok(status) => println!("상태 조회 성공: {:?}", status),

}
```

2. 플랫폼 상태 오류

```
use sev::error::{UserApiError, SevError};
```

```
match firmware.pek_generate() {
Err(UserApiError::SevError(SevError::InvalidPlatformState)) => {
println!("오류: 잘못된 플랫폼 상태");
println!("해결: 플랫폼을 리셋한 후 다시 시도하세요.");
// 플랫폼 리셋 시도
if let Err(reset_err) = firmware.platform_reset() {
println!("플랫폼 리셋 실패: {:?}", reset_err);
} else {
println!("플랫폼 리셋 성공 - 다시 시도하세요.");
}
}
Err(e) => println!("PEK 생성 오류: {:?}", e),
Ok(()) => println!("PEK 생성 성공"),
}
```

3. 펌웨어 버전 호환성 오류

```
use sev::firmware::host::{Build, Version};

let status = firmware.platform_status()?;
let minimum_version = Build {
version: Version { major: 0, minor: 14 },
build: 0,
```

```
};

if  status.build < minimum_version {
println!("오류: 펌웨어 버전이 너무 낮습니다.");
println!("현재 버전: {}.{}",
status.build.version.major,
status.build.version.minor);
println!("최소 요구 버전: {}.{}",
minimum_version.version.major,
minimum_version.version.minor);
println!("해결: 펌웨어를 업데이트하세요.");
}
```

4. 하드웨어 지원 확인

```
use std::fs;

// SEV 장치 파일 존재 확인
if !fs::metadata("/dev/sev").is_ok() {
println!("오류: SEV 장치가 없습니다.");
println!("해결: SEV 지원 하드웨어와 커널 모듈을 확인하세요.");
return Err("SEV 장치 없음".into());
}
```

```
// CPU 기능 확인
let cpuinfo = fs::read_to_string("/proc/cpuinfo")?;
if !cpuinfo.contains("sev") {
println!("경고: CPU에서 SEV 기능을 찾을 수 없습니다.");
println!("확인: AMD EPYC 프로세서와 BIOS SEV 설정을 확인하세요.");
}
```

초기화 상태 모니터링

초기화 진행 상황 추적

```
use std::time::{Duration, Instant};
use std::thread;

struct InitializationMonitor {
firmware: Firmware,
start_time: Instant,
}

impl InitializationMonitor {
fn new() -> Result<Self, UserApiError> {
Ok(InitializationMonitor {
```

```
firmware: Firmware::open()?,
start_time: Instant::now(),
})
}
fn monitor_initialization(&mut self) -> Result<(), UserApiError> {
loop {
let status = self.firmware.platform_status()?;
let elapsed = self.start_time.elapsed();
println!("[{:?}] 플랫폼 상태: {:?}", elapsed, status.state);
match status.state {
State::Initialized | State::Working => {
println!("초기화 완료! 총 소요 시간: {:?}", elapsed);
break;
}
State::Uninitialized => {
if elapsed > Duration::from_secs(300) { // 5분 타임아웃
println!("초기화 타임아웃 - 5분 초과");
break;
}
}
}
thread::sleep(Duration::from_secs(5));
}
```

```
Ok(())
}
}
```

베스트 프랙티스

1. 초기화 전 체크리스트

```
fn pre_initialization_check() -> Result<(), Box<dyn std::error::Error>> {
// 1. 하드웨어 지원 확인
if !std::path::Path::new("/dev/sev").exists() {
return Err("SEV 장치가 없습니다".into());
}
// 2. 권한 확인
let firmware = Firmware::open()?;
// 3. 펌웨어 버전 확인
let status = firmware.platform_status()?;
println!("펌웨어 버전 확인: {}.{}",
status.build.version.major,
status.build.version.minor);
// 4. 기존 게스트 확인
if status.guests > 0 {
println!("경고: {} 개의 게스트가 실행 중입니다", status.guests);
```

```
}
Ok(())
}
```

2. 안전한 초기화 절차

```
fn safe_initialization() -> Result<(), UserApiError> {
let mut firmware = Firmware::open()?;
// 백업을 위한 현재 상태 저장
let initial_status = firmware.platform_status()?;
println!("초기 상태 백업: {:?}", initial_status);
// 단계별 초기화 수행
let steps = [
("플랫폼 리셋", || firmware.platform_reset()),
("PEK 생성", || firmware.pek_generate()),
("PDH 생성", || firmware.pdh_generate()),
];
for (step_name, step_fn) in steps.iter() {
println!("{} 수행 중...", step_name);
match step_fn() {
Ok(()) => println!("{} 완료", step_name),
Err(e) => {
println!("{} 실패: {:?}", step_name, e);
```

```
return Err(e);
}
}
}
// 최종 검증
let final_status = firmware.platform_status()?;
if final_status.state == State::Initialized {
println!("초기화 성공적으로 완료");
} else {
println!("초기화 실패: 예상하지 못한 상태");
}
Ok(())
}
```

이 문서는 SEV 플랫폼 초기화의 전체 과정을 단계별로 설명하며, 실제 운영 환경에서 발생할 수 있는 다양한 상황에 대한 해결 방법을 제공합니다.